

## 1. Campus Events API

Manage university events with CRUD, filtering, and pagination.

Event(id, title, description, category, location, startTime, endTime, organizer, seatsTotal, seatsLeft)

*Endpoints:*

- GET /api/events?category=&from=&to=&page=&size= — filter + pagination
- GET /api/events/{id}
- POST /api/events — validate  $\text{endTime} > \text{startTime}$ ,  $\text{seatsLeft} \leq \text{seatsTotal}$
- PATCH /api/events/{id} — partial update
- DELETE /api/events/{id}
- POST /api/events/{id}/register — transaction-safe seat decrement

*Requirements:*

- DTO + Entity mapping
- Global exception handler
- JWT Authentication: Admin can create/update/delete; user can view/register
- Caching: Cache results of GET /api/events for 60s

*Tests & Docs:*

- Unit and integration tests
- Swagger / OpenAPI + Postman collection

## 2. BookSwap Marketplace

Enable users to exchange books with each other.

User(id, name, email)

Book(id, title, author, isbn, condition, ownerId, status[AVAILABLE, RESERVED, SWAPPED])

SwapRequest(id, bookId, requesterId, message, status[PENDING, ACCEPTED, REJECTED])

*Endpoints:*

- GET /api/books?title=&author=&status=&ownerId=
- POST /api/books — only owner can create
- PATCH /api/books/{id} — manage status transitions
- POST /api/swaps — create swap request
- PATCH /api/swaps/{id}/accept|reject — transactional status update
- GET /api/users/{id}/books — get all books by owner

*Requirements:*

- Soft delete or status-based management
- JWT Authentication with roles (USER / ADMIN)
- Validation: ISBN format check
- Partial search by title/author
- HATEOAS links (self, owner, swap)

*Tests & Docs:*

- Controller + repository tests

- Swagger + README (run instructions, env vars)

### 3. Smart Pantry (Inventory System)

Manage household inventory and generate restock/use recommendations.

Item(id, name, category, qty, minQty, unit, expiresAt, ownerId)

Recommendation(itemId, action[BUY, USE\_SOON], reason)

*Endpoints:*

- GET /api/items?expiringInDays=&category=
- POST /api/items — unique name per owner
- PATCH /api/items/{id} — adjust quantity
- GET /api/recommendations — auto-generated based on rules
- POST /api/items/bulk — CSV import (validate + partial success report)

*Requirements:*

- CSV parsing with detailed error reporting
- JWT Authentication (multi-tenant by owner)
- Caching: recommendations per owner
- Scheduler: recalculate daily using built-in scheduler

*Tests & Docs:*

- Unit tests for business rules
- Swagger + example requests

### 4. Course Planner & Syllabi API

Manage courses, modules, assignments, and syllabus versions.

Course(id, code, title, semester, instructorId)

Module(id, courseId, week, topic, reading)

Assignment(id, courseId, title, dueDate, points)

SyllabusVersion(id, courseId, version, createdAt, notes, isCurrent)

*Endpoints:*

- GET /api/courses?semester=&instructorId=
- POST /api/courses / PATCH /api/courses/{id}
- POST /api/courses/{id}/modules and GET /api/courses/{id}/modules
- POST /api/courses/{id}/assignments
- POST /api/courses/{id}/syllabus/versions — mark old as isCurrent=false
- GET /api/courses/{id}/syllabus/current and /versions

*Requirements:*

- Optimistic locking for syllabus updates
- Auth & Roles: INSTRUCTOR (CRUD), STUDENT (read-only)
- Validation: dueDate ≥ now, unique (course, week) for modules
- Performance: sorting and pagination

*Tests & Docs:*

- Versioning logic unit tests
- Swagger + sample seed data

## **5. Fitness Tracker Lite**

Track workouts and monitor weekly progress.

Workout(id, userId, type[RUN, BIKE, SWIM, GYM, YOGA], durationMin, distanceKm, calories, startedAt, notes)

Goal(id, userId, metric[DISTANCE, DURATION, SESSIONS], targetValue, period[WEEK, MONTH], startDate, endDate)

*Endpoints:*

- POST /api/workouts
- GET /api/workouts?from=&to=&type=
- GET /api/stats/weekly — aggregates duration, distance, sessions
- POST /api/goals and GET /api/goals/active
- GET /api/goals/progress — compute % progress server-side

*Requirements:*

- JWT Authentication: user can only access own data
- Validation: distance applies only to RUN/BIKE/SWIM
- Pagination and sorting by startedAt
- Unified error model (timestamp, path, code, message, details)

*Tests & Docs:*

- Unit tests for stats computation
- Swagger + example curl requests
- Basic CI pipeline (build + test)